

MODUL 13 & 14

NETWORKING dan DATA STORAGE (BAGIAN III)

1. Pada praktikum Modul 13 & 14, mahasiswa mempelajari implementasi konsep Networking dan Data Storage pada framework Flutter sebagai bagian penting dalam pengembangan aplikasi mobile modern. Kedua materi ini bertujuan untuk memberikan pemahaman mengenai bagaimana aplikasi dapat berkomunikasi dengan sumber data eksternal melalui jaringan internet serta bagaimana data tersebut dapat disimpan secara lokal agar tetap dapat digunakan meskipun perangkat tidak terhubung ke internet.

Pada bagian Networking, mahasiswa mempelajari penggunaan package Provider sebagai salah satu metode *state management* yang populer dalam Flutter. State management merupakan teknik yang digunakan untuk mengelola perubahan data dalam aplikasi sehingga antarmuka pengguna (user interface) dapat diperbarui secara otomatis ketika terjadi perubahan nilai pada data. Melalui penggunaan Provider, proses pengelolaan data menjadi lebih terstruktur karena data dapat diakses oleh berbagai widget tanpa perlu mengirimkan data secara berulang melalui parameter. Implementasi dilakukan dengan membuat kelas CounterProvider yang memanfaatkan ChangeNotifier untuk memberikan notifikasi kepada widget ketika nilai data berubah. Dengan mekanisme tersebut, aplikasi dapat menampilkan perubahan data secara real-time tanpa perlu melakukan refresh secara manual.

Praktikum networking diwujudkan dalam bentuk aplikasi Counter Application yang memiliki fungsi sederhana untuk menambah dan mengurangi nilai counter. Nilai counter disimpan dalam sebuah variabel yang dikelola oleh Provider. Ketika pengguna menekan tombol tambah atau kurang, aplikasi akan memanggil fungsi tertentu untuk memodifikasi nilai data, kemudian secara otomatis memperbarui tampilan melalui

mekanisme notifikasi yang disediakan oleh Provider. Melalui implementasi ini mahasiswa dapat memahami konsep dasar aliran data (*data flow*) antara model, provider, dan tampilan aplikasi.

Selain mempelajari state management, praktikum juga membahas implementasi Data Storage menggunakan database SQLite serta komunikasi data melalui API. Pada bagian ini digunakan beberapa package pendukung seperti http, sqflite, dan path. Package http berfungsi untuk melakukan komunikasi dengan server melalui protokol HTTP, sedangkan package sqflite digunakan untuk mengelola database SQLite yang tersimpan pada perangkat pengguna. Package path digunakan untuk membantu menentukan lokasi penyimpanan database secara sistematis pada sistem operasi perangkat.

Proses pengambilan data dilakukan melalui API publik yang menyediakan data pengguna dalam format JSON. Aplikasi mengirimkan permintaan (*request*) menggunakan metode GET, kemudian menerima respons dari server berupa sekumpulan data yang selanjutnya diproses menggunakan fungsi parsing JSON. Data yang telah diperoleh kemudian dikonversi menjadi objek model menggunakan kelas Character, sehingga data dapat dikelola dengan lebih terstruktur sesuai prinsip pemrograman berorientasi objek (*Object Oriented Programming*).

Pada sisi penyimpanan lokal, mahasiswa mengimplementasikan database SQLite dengan nama **characters.db** yang berisi tabel **characters**. Tabel tersebut menyimpan informasi pengguna berupa id, nama, dan username. Pengelolaan data dilakukan menggunakan konsep **CRUD (Create, Read, Update, Delete)**, meskipun pada praktikum ini implementasi yang digunakan lebih berfokus pada proses penyimpanan data, pembacaan data, dan penghapusan data. Data yang berhasil diambil dari API akan disimpan ke dalam database lokal sehingga dapat digunakan kembali tanpa harus

selalu melakukan permintaan ke server.

Integrasi antara API dan SQLite memberikan gambaran nyata mengenai mekanisme sinkronisasi data yang sering digunakan pada aplikasi mobile modern. Ketika pengguna melakukan proses sinkronisasi, data terbaru dari server akan diunduh dan disimpan ke database lokal. Selanjutnya data tersebut ditampilkan pada antarmuka aplikasi menggunakan widget seperti `ListView.builder` dan `ListTile`. Dengan demikian, aplikasi mampu menampilkan data secara dinamis sekaligus menjaga ketersediaan data ketika koneksi internet tidak tersedia.

Melalui praktikum ini, mahasiswa memperoleh pemahaman mengenai bagaimana membangun aplikasi Flutter yang mampu mengelola state secara efisien, melakukan komunikasi dengan server melalui API, serta menyimpan data secara lokal menggunakan SQLite. Kombinasi ketiga teknologi tersebut menjadi dasar penting dalam pengembangan aplikasi mobile yang responsif, terstruktur, dan mampu memberikan pengalaman pengguna yang lebih baik.

a. Code (boleh copy/screenshot dengan syarat terlihat jelas dan tidak blur)

➤ **main.dart**

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'character_model.dart';
import 'db_helper.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SQLite & HTTP Demo',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}

class CharacterListScreen extends StatefulWidget {
  const CharacterListScreen({super.key});

  @override
  State<CharacterListScreen> createState() => _CharacterListScreenState();
}

class _CharacterListScreenState extends State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }

  Future<void> _loadFromLocal() async {
    setState(() => _isLoading = true);
    final localData = await DatabaseHelper.instance.getAllCharacters();
    setState(() {
      _characters = localData;
      _isLoading = false;
    });
  }
}
```

```

// Fungsi Menghapus Semua Data dari SQLite
Future<void> _deleteAllData() async {
  bool? confirm = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Hapus Semua Data?'),
      content: const Text('apakah anda yakin?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Batal'),
        ),
        TextButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Hapus', style: TextStyle(color: Colors.red)),
        ),
      ],
    ),
  );

  if (confirm != true) return;

  setState(() => _isLoading = true);

  try {
    await DatabaseHelper.instance.clearTable();

    setState(() {
      _characters.clear();
    });

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Semua data berhasil dihapus dari sistem.'),
        ),
      );
    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Terjadi kesalahan: $e')));
    }
  } finally {
    setState(() => _isLoading = false);
  }
}

```

```

Future<void> _fetchFromAPI() async {
  setState(() => _isLoading = true);

  try {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/users'),
    );

    if (response.statusCode == 200) {
      final List<dynamic> dataJson = json.decode(response.body);

      await DatabaseHelper.instance.clearTable();

      List<Character> freshCharacters = [];

      for (var item in dataJson) {
        final char = Character.fromMap(item);
        freshCharacters.add(char);
        await DatabaseHelper.instance.insertCharacter(char);
      }

      setState(() => _characters = freshCharacters);

      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Berhasil sync data dari server!')),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('Gagal konek internet, pastikan online.'),
          ),
        );
      }
    } finally {
      setState(() => _isLoading = false);
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Ensiklopedia NPC'),
      actions: [
        IconButton(
          icon: const Icon(Icons.delete_forever, color: Colors.redAccent),

```

```

        tooltip: 'Hapus Semua Data',
        onPressed: _characters.isEmpty ? null : _deleteAllData,
      ),
      IconButton(
        icon: const Icon(Icons.cloud_download),
        tooltip: 'Sync dari Server',
        onPressed: _fetchFromAPI,
      ),
    ],
  ),
  body: _isLoading
    ? const Center(child: CircularProgressIndicator())
    : _characters.isEmpty
    ? const Center(child: Text('Data kosong.'))
    : ListView.builder(
        itemCount: _characters.length,
        itemBuilder: (context, index) {
          final char = _characters[index];
          return ListTile(
            leading: CircleAvatar(
              backgroundColor: Colors.deepPurple,
              child: Text(char.id.toString()),
            ),
            title: Text(char.name),
            subtitle: Text('Username: ${char.username}'),
          );
        },
      ),
);
}
}

```

➤ **db_helper.dart**

```

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import 'character_model.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('characters.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {

```

```

    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(path, version: 1, onCreate: _createDB);
  }

  Future _createDB(Database db, int version) async {
    await db.execute("""
      CREATE TABLE characters (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        username TEXT NOT NULL
      )
    """);
  }

  Future<void> insertCharacter(Character character) async {
    final db = await instance.database;
    await db.insert(
      'characters',
      character.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  Future<List<Character>> getAllCharacters() async {
    final db = await instance.database;
    final maps = await db.query('characters');

    return maps.map((map) => Character.fromMap(map)).toList();
  }

  Future<void> clearTable() async {
    final db = await instance.database;
    await db.delete('characters');
  }
}

```

➤ **character_model.dart**

```

class Character {
  final int id;
  final String name;
  final String username;

  Character({required this.id, required this.name, required this.username});

  factory Character.fromMap(Map<String, dynamic> map) {
    return Character(
      id: map['id'],
      name: map['name'],

```



```
        username: map['username'] ?? 'Unknown Username',
    );
}

Map<String, dynamic> toMap() {
    return {'id': id, 'name': name, 'username': username};
}
}
```

b. Screenshot output

1. NETWORKING



2. DATA STORAGE (BAGIAN III)



c. Penjelasan dari code dan output (korelasi)

Korelasi antara kode program dan output pada aplikasi Networking dapat dilihat dari bagaimana mekanisme Provider mengelola perubahan data yang terjadi pada aplikasi. Pada file pubspec.yaml, dependency provider ditambahkan agar aplikasi dapat menggunakan fitur state management yang disediakan oleh package tersebut. Setelah package berhasil diintegrasikan, pada file main.dart dibuat sebuah ChangeNotifierProvider yang berfungsi sebagai penyedia data utama bagi seluruh widget yang berada di dalam hierarki aplikasi. Objek yang dikelola oleh provider adalah CounterProvider, yaitu sebuah kelas yang mewarisi kemampuan dari ChangeNotifier. Kelas ini bertanggung jawab menyimpan nilai counter melalui variabel `_counter` dan menyediakan fungsi `increment()` serta `decrement()` untuk mengubah nilainya. Ketika salah satu fungsi tersebut dijalankan, nilai counter akan diperbarui sesuai aksi pengguna, kemudian method `notifyListeners()` dipanggil untuk mengirimkan notifikasi kepada seluruh widget yang sedang menggunakan data tersebut.

Pada tampilan aplikasi, nilai counter ditampilkan menggunakan widget `Consumer<CounterProvider>`. Widget ini secara otomatis mendengarkan

perubahan yang terjadi pada CounterProvider. Oleh karena itu, setiap kali nilai counter berubah, widget Consumer akan membangun ulang (*rebuild*) bagian tampilan yang menggunakan data tersebut sehingga angka yang muncul pada layar selalu sesuai dengan nilai terbaru.

Output yang ditampilkan menunjukkan bahwa ketika aplikasi pertama kali dijalankan, nilai counter berada pada angka 0 sesuai dengan nilai awal variabel `_counter`. Saat pengguna menekan tombol tambah (+), nilai counter bertambah satu per satu dan langsung diperbarui pada layar. Sebaliknya, ketika tombol kurang (-) ditekan, nilai counter akan berkurang selama nilainya masih lebih besar dari nol. Proses ini terjadi secara real-time tanpa memerlukan refresh halaman karena Provider secara otomatis mengelola sinkronisasi antara data dan antarmuka pengguna.

Berdasarkan hasil output tersebut dapat disimpulkan bahwa implementasi Provider berhasil menerapkan konsep reactive programming, yaitu tampilan aplikasi akan selalu mengikuti perubahan data yang terjadi pada sumber data utama. Hal ini membuat aplikasi menjadi lebih efisien, mudah dikembangkan, dan lebih terstruktur dibandingkan pengelolaan state secara manual.

DATA STORAGE (Bagian III)

korelasi antara kode program dan output pada aplikasi Data Storage menunjukkan implementasi yang terintegrasi antara proses pengambilan data dari server dan penyimpanan data secara lokal menggunakan SQLite. Pada tahap konfigurasi, package http digunakan untuk berkomunikasi dengan API, package sqflite digunakan sebagai database lokal, sedangkan package path digunakan untuk menentukan lokasi penyimpanan database pada perangkat.

Ketika aplikasi dijalankan, method `initState()` secara otomatis memanggil fungsi `_loadFromLocal()`. Fungsi ini bertugas membaca seluruh data yang tersimpan pada database SQLite melalui method `getAllCharacters()` yang terdapat pada kelas `DatabaseHelper`. Jika database belum memiliki data, maka list `_characters` akan kosong sehingga aplikasi menampilkan pesan "**Data kosong**" pada layar. Tampilan tersebut menunjukkan bahwa aplikasi berhasil melakukan pengecekan terhadap isi database lokal sebelum menampilkan data kepada pengguna.

Proses sinkronisasi data dilakukan melalui tombol **Sync dari Server** yang terdapat pada AppBar. Ketika tombol tersebut ditekan, fungsi `_fetchFromAPI()` akan dijalankan. Fungsi ini mengirimkan permintaan HTTP GET ke API dan menerima respons berupa data JSON. Setelah data berhasil diterima, aplikasi melakukan parsing JSON dan mengubah setiap data menjadi objek `Character` menggunakan method `fromMap()`. Selanjutnya data tersebut disimpan ke database SQLite melalui method `insertCharacter()`.

Hasil dari proses penyimpanan dapat langsung dilihat pada output aplikasi. Data

yang sebelumnya kosong berubah menjadi daftar pengguna yang ditampilkan menggunakan widget `ListView.builder`. Setiap item ditampilkan dalam bentuk `ListTile` yang memuat informasi id, nama, dan username. Tampilan ini menunjukkan bahwa data yang diperoleh dari server telah berhasil diproses, disimpan, dan ditampilkan kembali kepada pengguna melalui database lokal.

Selain proses penyimpanan data, aplikasi juga menyediakan fitur penghapusan seluruh data melalui tombol **Hapus Semua Data**. Ketika tombol ini dipilih dan pengguna memberikan konfirmasi, method `clearTable()` akan menghapus seluruh isi tabel `characters` pada `SQLite`. Setelah proses selesai, daftar data yang sebelumnya ditampilkan akan hilang dan aplikasi kembali menampilkan pesan "**Data kosong**". Perubahan output ini membuktikan bahwa operasi penghapusan data pada database telah berjalan dengan benar dan hasilnya langsung tercermin pada tampilan aplikasi.

Secara keseluruhan, output yang dihasilkan sesuai dengan alur kerja yang telah dirancang pada kode program. Data yang diperoleh dari API dapat disimpan ke `SQLite`, ditampilkan kembali pada antarmuka aplikasi, serta dihapus sesuai kebutuhan pengguna.

Implementasi ini menunjukkan bahwa integrasi antara layanan web (API), model data, database lokal, dan antarmuka pengguna telah berjalan dengan baik. Dengan adanya mekanisme tersebut, aplikasi mampu menyediakan data secara lebih cepat, mengurangi ketergantungan terhadap koneksi internet, serta meningkatkan efisiensi dalam pengelolaan informasi pada perangkat mobile.